



A parallel approach for solving the Green's function based on the nested-dissection and selected-inverse methods

Emmanuel Agullo, Alfredo Buttari, **Matthieu Robeyns**

HPCSIM25, September 11, 2025



- Energy-oriented Centre of Excellence for Exascale HPC applications (EoCoE) III.
- European collaboration project between :
 - **French** : CNRS+Inria team : Emmanuel Agullo, Alfredo Buttari, Matthieu Robeyns.
 - **Germany** : Jülich team : Edoardo Di Napoli, Christoph Conrads, and Gustavo Ramirez Hidalgo.
 - **Italy** : CNR team : Alessandro Pecchia, and Flavia Carafone.
- Develop application on **quantum transport** with new circuits.

Pb : New circuits tend to be **more challenging regarding time and memory consumption**.

Study case : Non-Equilibrium Green's function

- Example with Non-Equilibrium Green's Function (NEGF) equations :

$$\begin{cases} [ES(k) - H(k) - \Sigma^r(k, E)]G^r(k, E) = I & \text{(Dyson equation)} \\ G^n(k, E) = G^r(k, E)\Sigma^n(k, E)G^{r\dagger}(k, E) & \text{(Keldysh equation)} \end{cases}$$

Study case : Non-Equilibrium Green's function

- Example with Non-Equilibrium Green's Function (NEGF) equations :

$$\begin{cases} [ES(k) - H(k) - \Sigma^r(k, E)]G^r(k, E) = I & \text{(Dyson equation)} \\ G^n(k, E) = G^r(k, E)\Sigma^n(k, E)G^{r\dagger}(k, E) & \text{(Keldysh equation)} \end{cases}$$

- From Dyson equation :

- $ES(k)$: Energy shift,
- $H(k)$: Hamiltonian,
- $\Sigma^r(k, E)$: Self-energy,
- $G^r(k, E)$: retarded Green's function,
- I : Identity matrix.

- From Keldysh equation :

- $G^n(k, E)$: non-equilibrium Green's function,
- $\Sigma^n(k, E)$: non-equilibrium self-energies,
- $G^{r\dagger}$: advanced Green's function, $\dagger$ is the adjoint notation.

Study case : Non-Equilibrium Green's function

- Example with Non-Equilibrium Green's Function (NEGF) equations :

$$\begin{cases} [ES(k) - H(k) - \Sigma^r(k, E)]G^r(k, E) = I & \text{(Dyson equation)} \\ G^n(k, E) = G^r(k, E)\Sigma^n(k, E)G^{r\dagger}(k, E) & \text{(Keldysh equation)} \end{cases}$$

- From Dyson equation :

- $ES(k)$: Energy shift,
- $H(k)$: Hamiltonian,
- $\Sigma^r(k, E)$: Self-energy,
- $G^r(k, E)$: retarded Green's function,
- I : Identity matrix.

- From Keldysh equation :

- $G^n(k, E)$: non-equilibrium Green's function,
- $\Sigma^n(k, E)$: non-equilibrium self-energies,
- $G^{r\dagger}$: advanced Green's function, $\dagger$ is the adjoint notation.

- **Unknowns** : $G^r(k, E)$, and $G^n(k, E)$.

Dyson Equation

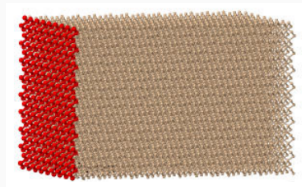
- Dyson equation :

$$[ES(k) - H(k) - \Sigma^r(k, E)]G^r(k, E) = I$$

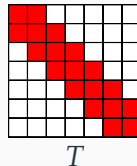
Dyson equation

- Dyson equation :

$$[ES(k) - H(k) - \Sigma^r(k, E)]G^r(k, E) = I \Leftrightarrow T(k, E)G^r(k, E) = I$$



rep.
→



□ : empty block

■ : dense block

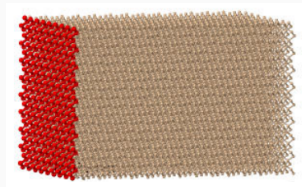
Quantum circuit (left) represented in tridiagonal matrix T (right).

- Consider $T(k, E)$ is a block tridiagonal matrix.

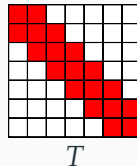
Dyson equation

- Dyson equation :

$$[ES(k) - H(k) - \Sigma^r(k, E)]G^r(k, E) = I \Leftrightarrow T(k, E)G^r(k, E) = I$$



rep.
→



□ : empty block

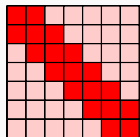
■ : dense block

Quantum circuit (left) represented in tridiagonal matrix T (right).

- Consider $T(k, E)$ is a block tridiagonal matrix.
- $G^r(k, E) = T^{-1}(k, E)$?

Dyson equation : Find $G^r(k, e)$

 Compute $T^{-1}(k, E)$, because $T^{-1}(k, E)$ is dense.



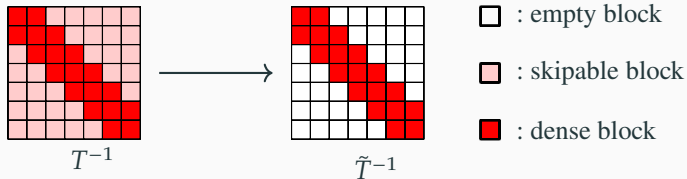
T^{-1}

 : empty block

 : dense block

Dyson equation : Find $G^r(k, e)$

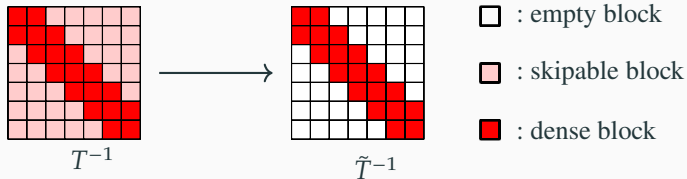
👉 Compute $T^{-1}(k, E)$, because $T^{-1}(k, E)$ is dense.



👍 NEGF application only need tridiagonals block of $T^{-1}(k, E)$.

Dyson equation : Find $G^r(k, e)$

🗨️ Compute $T^{-1}(k, E)$, because $T^{-1}(k, E)$ is dense.



👍 NEGF application only need tridiagonals block of $T^{-1}(k, E)$.

👍 Can avoid off-diagonal computation $\Rightarrow$ related work : Block Recursive¹, Block-Cyclic Reduction².

- Develop a **new framework** with parallelism thanks to **Selected Inverse (SI)** algorithm.

1. Recursive Block Coding - A New Approach to Transform Coding, Farrelle et Jain, 1986

2. Cyclic reduction-history and applications, Gander et Golub, 1998

Selected Inverse ^{*a*}

-
- a.* SelInv—An Algorithm for Selected Inversion of a Sparse Symmetric Matrix, 2011

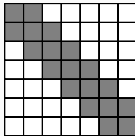
Algorithm : LU factorization of T

Input : T ::Matrix

Output : T ::Matrix (factor form)

```
1  $n \leftarrow \text{size}(T, 1)$ 
2 for  $i$  in  $1 : n-1$  do
3    $T_{i,i} \leftarrow \text{lu}(T_{i,i})$                                      // Step 1 : create  $L_{i,i}U_{i,i}$ 
4   if  $T_{i,i+1} \neq 0$  then
5      $T_{i+1,i} \leftarrow T_{i+1,i}U_{i,i}^{-1}$                        // Step 2 : Update  $L_{i+1:nb,i}$ 
6      $T_{i,i+1} \leftarrow L_{i,i}^{-1}T_{i,i+1}$                      // Step 3 : Update  $U_{i,i+1:nb}$ 
7    $T_{i+1,i+1} \leftarrow -T_{i+1,i}T_{i,i+1}$                      // Step 4 : Update  $T_{i+1:nb,i+1:nb}$ 
```

■ : Dense block □ : Zeros block



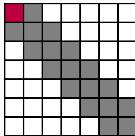
Algorithm : LU factorization of T

Input : T ::Matrix

Output : T ::Matrix (factor form)

```
1  $n \leftarrow \text{size}(T, 1)$ 
2 for  $i$  in  $1 : n-1$  do
3    $T_{i,i} \leftarrow \text{lu}(T_{i,i})$  // Step 1 : create  $L_{i,i}U_{i,i}$ 
4   if  $T_{i,i+1} \neq 0$  then
5      $T_{i+1,i} \leftarrow T_{i+1,i}U_{i,i}^{-1}$  // Step 2 : Update  $L_{i+1:nb,i}$ 
6      $T_{i,i+1} \leftarrow L_{i,i}^{-1}T_{i,i+1}$  // Step 3 : Update  $U_{i,i+1:nb}$ 
7    $T_{i+1,i+1} \leftarrow -T_{i+1,i}T_{i,i+1}$  // Step 4 : Update  $T_{i+1:nb,i+1:nb}$ 
```

■ : Dense block □ : Zeros block ■ : LU factor block



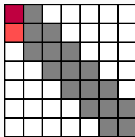
Algorithm : LU factorization of T

Input : T ::Matrix

Output : T ::Matrix (factor form)

```
1  $n \leftarrow \text{size}(T, 1)$ 
2 for  $i$  in  $1 : n-1$  do
3    $T_{i,i} \leftarrow \text{lu}(T_{i,i})$                                      // Step 1 : create  $L_{i,i}U_{i,i}$ 
4   if  $T_{i,i+1} \neq 0$  then
5      $T_{i+1,i} \leftarrow T_{i+1,i}U_{i,i}^{-1}$                        // Step 2 : Update  $L_{i+1:nb,i}$ 
6      $T_{i,i+1} \leftarrow L_{i,i}^{-1}T_{i,i+1}$                      // Step 3 : Update  $U_{i,i+1:nb}$ 
7    $T_{i+1,i+1} \leftarrow -T_{i+1,i}T_{i,i+1}$                      // Step 4 : Update  $T_{i+1:nb,i+1:nb}$ 
```

■ : Dense block □ : Zeros block ■ : LU factor block
■ : L factor block



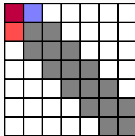
Algorithm : LU factorization of T

Input : T ::Matrix

Output : T ::Matrix (factor form)

```
1  $n \leftarrow \text{size}(T, 1)$ 
2 for  $i$  in  $1 : n-1$  do
3    $T_{i,i} \leftarrow \text{lu}(T_{i,i})$                                      // Step 1 : create  $L_{i,i}U_{i,i}$ 
4   if  $T_{i,i+1} \neq 0$  then
5      $T_{i+1,i} \leftarrow T_{i+1,i}U_{i,i}^{-1}$                        // Step 2 : Update  $L_{i+1:nb,i}$ 
6      $T_{i,i+1} \leftarrow L_{i,i}^{-1}T_{i,i+1}$                      // Step 3 : Update  $U_{i,i+1:nb}$ 
7    $T_{i+1,i+1} \leftarrow -T_{i+1,i}T_{i,i+1}$                      // Step 4 : Update  $T_{i+1:nb,i+1:nb}$ 
```

■ : Dense block □ : Zeros block ■ : LU factor block
■ : L factor block ■ : U factor block



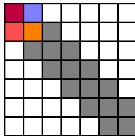
Algorithm : LU factorization of T

Input : T ::Matrix

Output : T ::Matrix (factor form)

```
1  $n \leftarrow \text{size}(T, 1)$ 
2 for  $i$  in  $1 : n-1$  do
3    $T_{i,i} \leftarrow \text{lu}(T_{i,i})$  // Step 1 : create  $L_{i,i}U_{i,i}$ 
4   if  $T_{i,i+1} \neq 0$  then
5      $T_{i+1,i} \leftarrow T_{i+1,i}U_{i,i}^{-1}$  // Step 2 : Update  $L_{i+1:nb,i}$ 
6      $T_{i,i+1} \leftarrow L_{i,i}^{-1}T_{i,i+1}$  // Step 3 : Update  $U_{i,i+1:nb}$ 
7    $T_{i+1,i+1} \leftarrow -T_{i+1,i}T_{i,i+1}$  // Step 4 : Update  $T_{i+1:nb,i+1:nb}$ 
```

■ : Dense block □ : Zeros block ■ : LU factor block
■ : L factor block ■ : U factor block ■ : Updated block



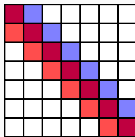
Algorithm : LU factorization of T

Input : T ::Matrix

Output : T ::Matrix (factor form)

```
1  $n \leftarrow \text{size}(T, 1)$ 
2 for  $i$  in  $1 : n-1$  do
3    $T_{i,i} \leftarrow \text{lu}(T_{i,i})$                                      // Step 1 : create  $L_{i,i}U_{i,i}$ 
4   if  $T_{i,i+1} \neq 0$  then
5      $T_{i+1,i} \leftarrow T_{i+1,i}U_{i,i}^{-1}$                        // Step 2 : Update  $L_{i+1:nb,i}$ 
6      $T_{i,i+1} \leftarrow L_{i,i}^{-1}T_{i,i+1}$                      // Step 3 : Update  $U_{i,i+1:nb}$ 
7    $T_{i+1,i+1} \leftarrow -T_{i+1,i}T_{i,i+1}$                      // Step 4 : Update  $T_{i+1:nb,i+1:nb}$ 
```

■ : Dense block □ : Zeros block ■ : LU factor block
■ : L factor block ■ : U factor block ■ : Updated block



Let $T \in \mathbb{R}^{m \times m}$, n diagonal blocks of size q .

- **Cost :** $\mathcal{O}(n(\frac{11}{3}q^3 + q^2))$
- **Memory :** $\mathcal{O}((3n - 2)q^2)$

Selected Inverse : Algorithm

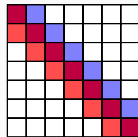
Algorithm : Selected inverse of T

Input : T ::Matrix (in LU format)

Output : T ::Matrix (in inverse format)

```
1  $n \leftarrow \text{size}(T, 1)$ 
2 for  $i$  in  $n : -1 : 1$  do
3    $T_{i+1,i} \leftarrow -T_{i+1,i+1} T_{i+1,i} L_{i,i}^{-1}$            // Step 1 : Compute  $T_{i+1,i}$ 
4    $T_{i,i+1} \leftarrow -U_{i,i}^{-1} T_{i,i+1} T_{i+1,i+1}$        // Step 2 : Compute  $T_{i,i+1}$ 
5    $T_{i,i} \leftarrow U_{i,i}^{-1} L_{i,i}^{-1} + T_{i,i+1} L_{i+1,i}^{-1}$  // Step 3 : Update  $T_{i,i}$ 
```

■ : Dense block □ : Zeros block ■ : LU factor block
■ : L factor block ■ : U factor block ■ : Updated block



Selected Inverse : Algorithm

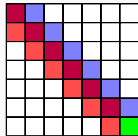
Algorithm : Selected inverse of T

Input : T ::Matrix (in LU format)

Output : T ::Matrix (in inverse format)

```
1  $n \leftarrow \text{size}(T, 1)$ 
2 for  $i$  in  $n : -1 : 1$  do
3    $T_{i+1,i} \leftarrow -T_{i+1,i+1}T_{i+1,i}L_{i,i}^{-1}$            // Step 1 : Compute  $T_{i+1,i}$ 
4    $T_{i,i+1} \leftarrow -U_{i,i}^{-1}T_{i,i+1}T_{i+1,i+1}$        // Step 2 : Compute  $T_{i,i+1}$ 
5    $T_{i,i} \leftarrow U_{i,i}^{-1}L_{i,i}^{-1} + T_{i,i+1}L_{i+1,i}^{-1}$  // Step 3 : Update  $T_{i,i}$ 
```

■ : Dense block □ : Zeros block ■ : LU factor block
■ : L factor block ■ : U factor block ■ : Updated block
■ : Inversed block



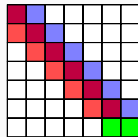
Algorithm : Selected inverse of T

Input : T ::Matrix (in LU format)

Output : T ::Matrix (in inverse format)

```
1  $n \leftarrow \text{size}(T, 1)$ 
2 for  $i$  in  $n : -1 : 1$  do
3    $T_{i+1,i} \leftarrow -T_{i+1,i+1}T_{i+1,i}L_{i,i}^{-1}$  // Step 1 : Compute  $T_{i+1,i}$ 
4    $T_{i,i+1} \leftarrow -U_{i,i}^{-1}T_{i,i+1}T_{i+1,i+1}$  // Step 2 : Compute  $T_{i,i+1}$ 
5    $T_{i,i} \leftarrow U_{i,i}^{-1}L_{i,i}^{-1} + T_{i,i+1}L_{i+1,i}^{-1}$  // Step 3 : Update  $T_{i,i}$ 
```

■ : Dense block □ : Zeros block ■ : LU factor block
■ : L factor block ■ : U factor block ■ : Updated block
■ : Inversed block



Selected Inverse : Algorithm

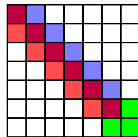
Algorithm : Selected inverse of T

Input : T ::Matrix (in LU format)

Output : T ::Matrix (in inverse format)

```
1  $n \leftarrow \text{size}(T, 1)$ 
2 for  $i$  in  $n : -1 : 1$  do
3    $T_{i+1,i} \leftarrow -T_{i+1,i+1}T_{i+1,i}L_{i,i}^{-1}$  // Step 1 : Compute  $T_{i+1,i}$ 
4    $T_{i,i+1} \leftarrow -U_{i,i}^{-1}T_{i,i+1}T_{i+1,i+1}$  // Step 2 : Compute  $T_{i,i+1}$ 
5    $T_{i,i} \leftarrow U_{i,i}^{-1}L_{i,i}^{-1} + T_{i,i+1}L_{i+1,i}^{-1}$  // Step 3 : Update  $T_{i,i}$ 
```

■ : Dense block □ : Zeros block ■ : LU factor block
■ : L factor block ■ : U factor block ■ : Updated block
■ : Inversed block



Selected Inverse : Algorithm

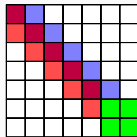
Algorithm : Selected inverse of T

Input : T ::Matrix (in LU format)

Output : T ::Matrix (in inverse format)

```
1  $n \leftarrow \text{size}(T, 1)$ 
2 for  $i$  in  $n : -1 : 1$  do
3    $T_{i+1,i} \leftarrow -T_{i+1,i+1}T_{i+1,i}L_{i,i}^{-1}$  // Step 1 : Compute  $T_{i+1,i}$ 
4    $T_{i,i+1} \leftarrow -U_{i,i}^{-1}T_{i,i+1}T_{i+1,i+1}$  // Step 2 : Compute  $T_{i,i+1}$ 
5    $T_{i,i} \leftarrow U_{i,i}^{-1}L_{i,i}^{-1} + T_{i,i+1}L_{i+1,i}^{-1}$  // Step 3 : Update  $T_{i,i}$ 
```

■ : Dense block □ : Zeros block ■ : LU factor block
■ : L factor block ■ : U factor block ■ : Updated block
■ : Inversed block



Selected Inverse : Algorithm

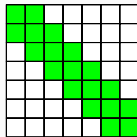
Algorithm : Selected inverse of T

Input : T ::Matrix (in LU format)

Output : T ::Matrix (in inverse format)

```
1  $n \leftarrow \text{size}(T, 1)$ 
2 for  $i$  in  $n : -1 : 1$  do
3    $T_{i+1,i} \leftarrow -T_{i+1,i+1} T_{i+1,i} L_{i,i}^{-1}$            // Step 1 : Compute  $T_{i+1,i}$ 
4    $T_{i,i+1} \leftarrow -U_{i,i}^{-1} T_{i,i+1} T_{i+1,i+1}$        // Step 2 : Compute  $T_{i,i+1}$ 
5    $T_{i,i} \leftarrow U_{i,i}^{-1} L_{i,i}^{-1} + T_{i,i+1} L_{i+1,i}^{-1}$  // Step 3 : Update  $T_{i,i}$ 
```

■ : Dense block □ : Zeros block ■ : LU factor block
■ : L factor block ■ : U factor block ■ : Updated block
■ : Inversed block



Let $T \in \mathbb{R}^{m \times m}$, n diagonal blocks of size q .

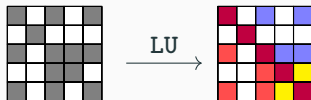
- **Cost** : $\mathcal{O}(n(4q^3 + q^2) - (q^3 + q^2))$
- **Memory** : $\mathcal{O}((3n - 2)q^2)$

- LU+SI can be easily generalized for any type of matrix.

- LU+SI can be easily generalized for any type of matrix.

- LU : **Fill-in** effect.

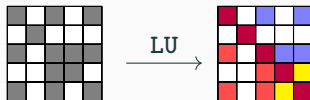
■ : Dense block □ : Zeros block ■ : LU factor block
■ : L factor block ■ : U factor block ■ : Fill-in block



- LU+SI can be easily generalized for any type of matrix.

- LU : **Fill-in** effect.

■ : Dense block □ : Zeros block ■ : LU factor block
■ : L factor block ■ : U factor block ■ : Fill-in block

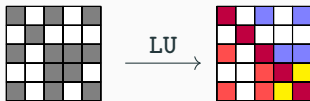


- SI : No additional fill in ; but keep same structure as LU factors.

- LU+SI can be easily generalized for any type of matrix.

- LU : **Fill-in** effect.

■ : Dense block □ : Zeros block ■ : LU factor block
■ : L factor block ■ : U factor block ■ : Fill-in block



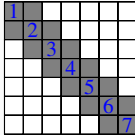
- SI : No additional fill in ; but keep same structure as LU factors.

👍 Convenient framework to investigate **parallelism**.

Parallelism

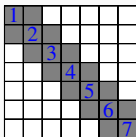
How to extract parallelilism

■ : Dense block □ : Zeros block



How to extract parallelism

■ : Dense block □ : Zeros block

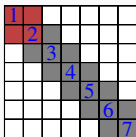


node
→



How to extract parallelilism

■ : Dense block □ : Zeros block

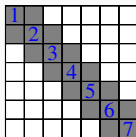


node
→



How to extract parallelilism

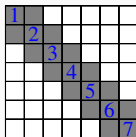
■ : Dense block □ : Zeros block



- Computation order :
 - LU : Block 1 to 7.

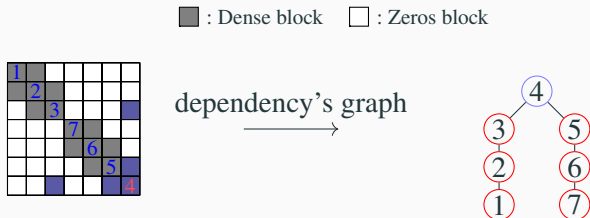
How to extract parallelism

■ : Dense block □ : Zeros block



- Computation order :
 - LU : Block 1 to 7.
 - SI : Block 7 to 1.

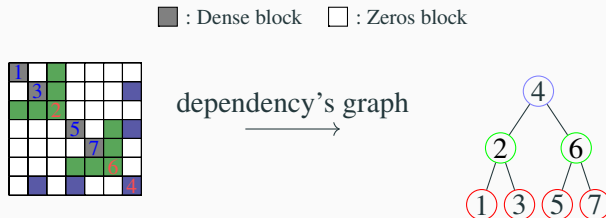
How to extract parallelism



- Computation order :

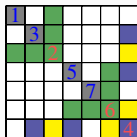
- LU : Block 1 to 4.
- SI : Block 4 to 1.

👍 Branch 1 – 2 – 3 and 7 – 6 – 5 can be done in parallel !

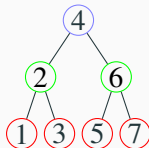


- **Nested-dissection**³ : Recursively reordering the matrix.

■ : Dense block □ : Zeros block ■ : Fill-in block



dependency's graph
→



- **Nested-dissection**³ : Recursively reordering the matrix.

👍 Increase the degree of parallelism.

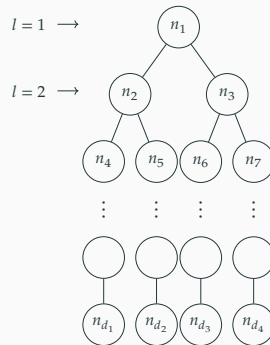
👎 Add **fill-in** effect $\Rightarrow$ increase the computation cost.

Idea : Find the **good trade-off** between **fill-in** and **parallelization**.

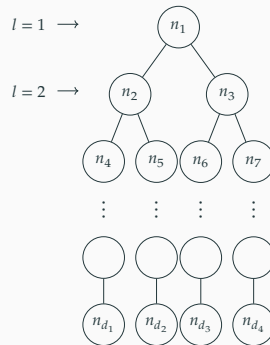
Model

- p threads :

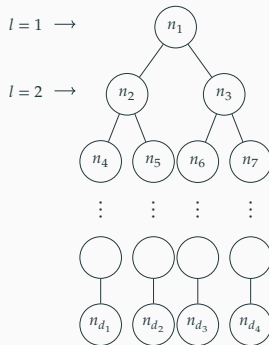
- $p = 4 \rightarrow$ optimal !
- $p > 4 \rightarrow$ too much $\Rightarrow$ increase l .
- $p < 4 \rightarrow$ not enough $\Rightarrow$ decrease l .



- p threads :
 - $p = 4 \rightarrow$ optimal !
 - $p > 4 \rightarrow$ too much $\Rightarrow$ increase l .
 - $p < 4 \rightarrow$ not enough $\Rightarrow$ decrease l .
- Find l for p threads : $l = \lceil \log_2(p) \rceil$



- p threads :
 - $p = 4 \rightarrow$ optimal !
 - $p > 4 \rightarrow$ too much $\Rightarrow$ increase l .
 - $p < 4 \rightarrow$ not enough $\Rightarrow$ decrease l .
- Find l for p threads : $l = \lceil \log_2(p) \rceil$
- Execution is linearly depending on the depth of the tree $d = k + l$, where k constant.



Remark

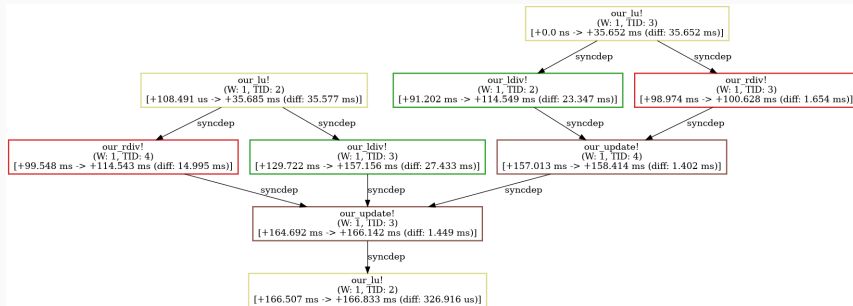
Let d -depth tree with n nodes of size b :

- Total computation : $(\frac{17}{3}n - \frac{12}{3}d - \frac{71}{3})b^3$
- Critical path : $(\frac{17}{3}d - 7)b^3$

Implementation

Task-based model

- Consider line computation as **task**.
- Task execute when they have no dependency with other task.
- 👍 Execution of **independent** task can be done in parallel.
- Represented as Directed Acyclic Graph (DAG), where :
 - nodes = task (elementary operation),
 - edges = dependancies.



Sequential task flow

- **Sequential task flow (STF)** : automatically generate the DAG of task and schedule task to be execute at the runtime.
- Keyword for dependency control : InOut (read/write), In (read only), Out (write only).



Easy to implement. Minor modification from sequential code.

Algorithm : LU factorization of T

Input : T ::Matrix

Output : T ::Matrix (factor form)

```
1  $n \leftarrow \text{size}(T, 1)$ 
2 for  $i$  in  $1 : n-1$  do
3     @spwan(InOut( $T_{i,i}$ ), lu)                                // Step 1 : create  $L_{i,i}U_{i,i}$ 
4     for  $j$  in  $i+1 : n$  when  $T_{i,j} \neq 0$  do
5         | @spwan(InOut( $T_{j,i}$ ), In( $U_{i,i}$ ), /)                // Step 2 : Update  $L_{i+1:nb,i}$ 
6         | @spwan(InOut( $T_{i,j}$ ), In( $L_{i,i}$ ), \)                // Step 3 : Update  $U_{i,i+1:nb}$ 
7     for  $k$  in index of intersect  $L_{i+1:n,i}, U_{i,i+1:n}$  do
8         | @spwan(InOut( $T_k$ ), In( $T_{k[1],i}, T_{i,k[2]}$ ),  $\{-, \text{prod}\}$ ) // Step 4 : Update  $T_{i+1:nb,i+1:nb}$ 
```

Sequential task flow

- **Sequential task flow (STF)** : automatically generate the DAG of task and schedule task to be execute at the runtime.
- Keyword for dependency control : InOut (read/write), In (read only), Out (write only).
- 👍 Easy to implement. Minor modification from sequential code.

Algorithm : Selected inverse of T

Input : T ::Matrix (in LU format)

Output : T ::Matrix (in inverse format)

```
1  $n \leftarrow \text{size}(T, 1)$ 
2 for  $i$  in  $n : -1 : 1$  do
3      $idx \leftarrow \text{intersect}(T_{i+1:n,i}, T_{i+1:n,i+1:n})$  // Step 0 : couple of non zero block
4     for  $j$  in  $i + 1 : n$  do
5         @spwan(InOut( $T_{i+1:n,i}$ ), In( $T_{j,idx}, T_{idx,i}, L_{i,i}$ ),  $\{-, \text{prod}, /\}$ ) // Step 1 : Compute  $T_{i+1:nb,i}$ 
6         @spwan(InOut( $T_{i,i+1:n}$ ), In( $U_{i,i}, T_{i,idx}, T_{idx,j}$ ),  $\{-, \backslash, \text{prod}\}$ ) // Step 2 : Compute  $T_{i,i+1:n}$ 
7     @spwan(InOut( $T_{i,i}$ ), In( $U_{i,i}^{-1} L_{i,i}^{-1}, T_{i,i+1:n}, L_{i+1:n,i}$ ),  $\{+, /\}$ ) // Step 3 : Update  $T_{i,i}$ 
```

Experiments

- Language code : Julia.
- Task-based library : Dagger⁴.
- Cluster : PlafRIM⁵.
- Result only LU factorization.
- Still **work in progress**.

4. <https://juliaparallel.org/Dagger.jl/dev/>

5. <https://www.plafrim.fr/>

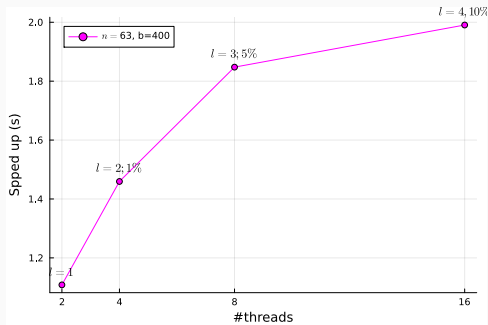


Figure 2 – Performance with multiple threads.

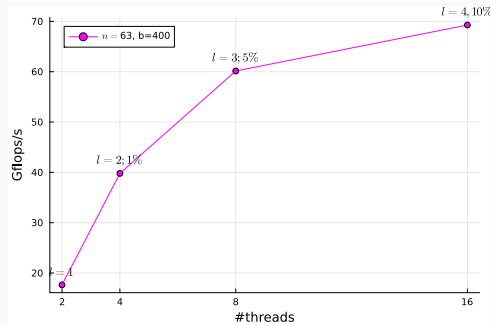


Figure 3 – Gflops/s with multiple threads.

- **Legend :** n = number of diagonal blocks, b = size of blocks.



- Collaboration with one of main developers (Julian Samaroo).
- Different times for a kernel execution. (Fixed)
- Overhead between task execution. (W.I.P.)
- Management of large number of task. (W.I.P.)

Contributions

- Reformulate G' in the framework LU+ SI.
- Exploit Nested Dissection for parallelism and study model.
- Create prototype based on STF.

Perspectives

- Add a fine-grained parallelism by blocks.
- Using another task-model library (Chameleon and StarPU).
- Compress block with low-rank approximation.
- Extend approach to Keldysh's equation (G^n).